

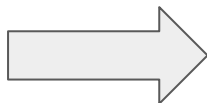
CX4240 Spring 2026

(Large) Language Model (Part II): Post-Training

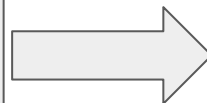
Bo Dai
School of CSE, Georgia Tech
bodai@cc.gatech.edu

(Large) Language Models

Training Data



Learning Algorithm



Target Function



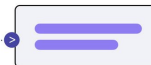
Text Input

Summarize
this article:



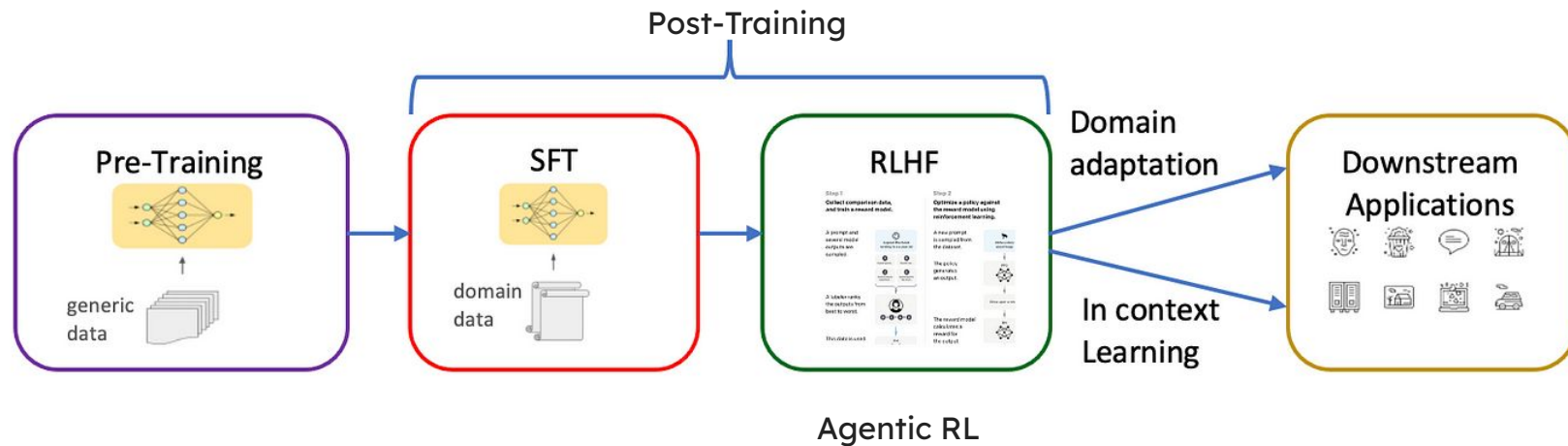
Text Generation Model
(Large Language Model)

Output

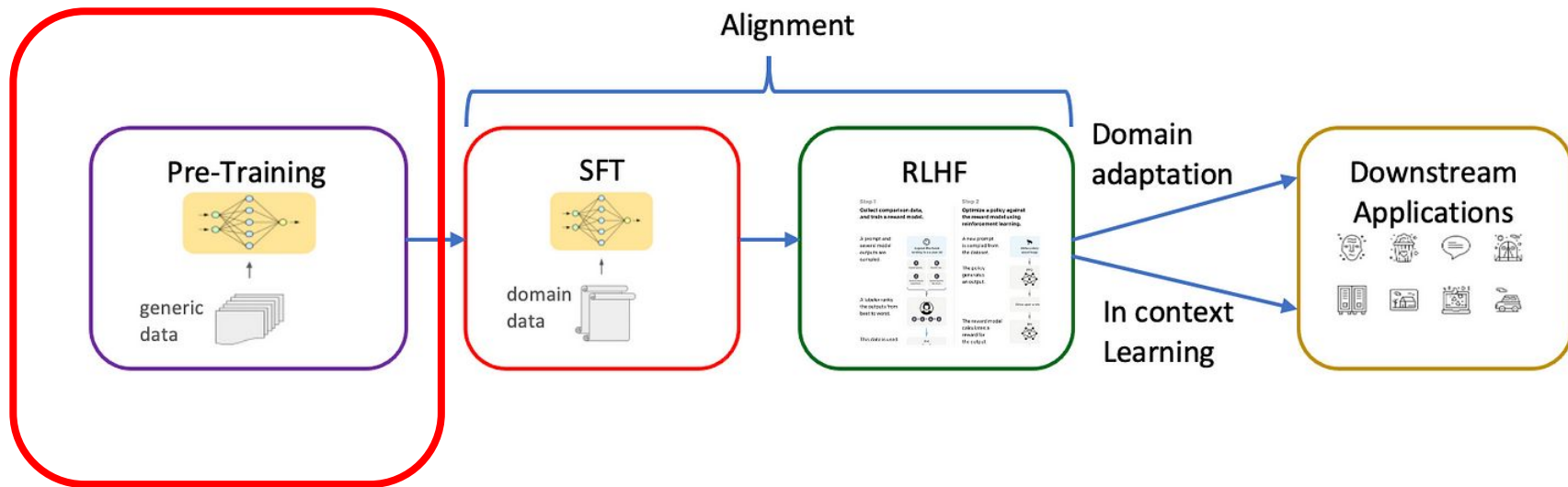


Unlabelled Data: text sequences

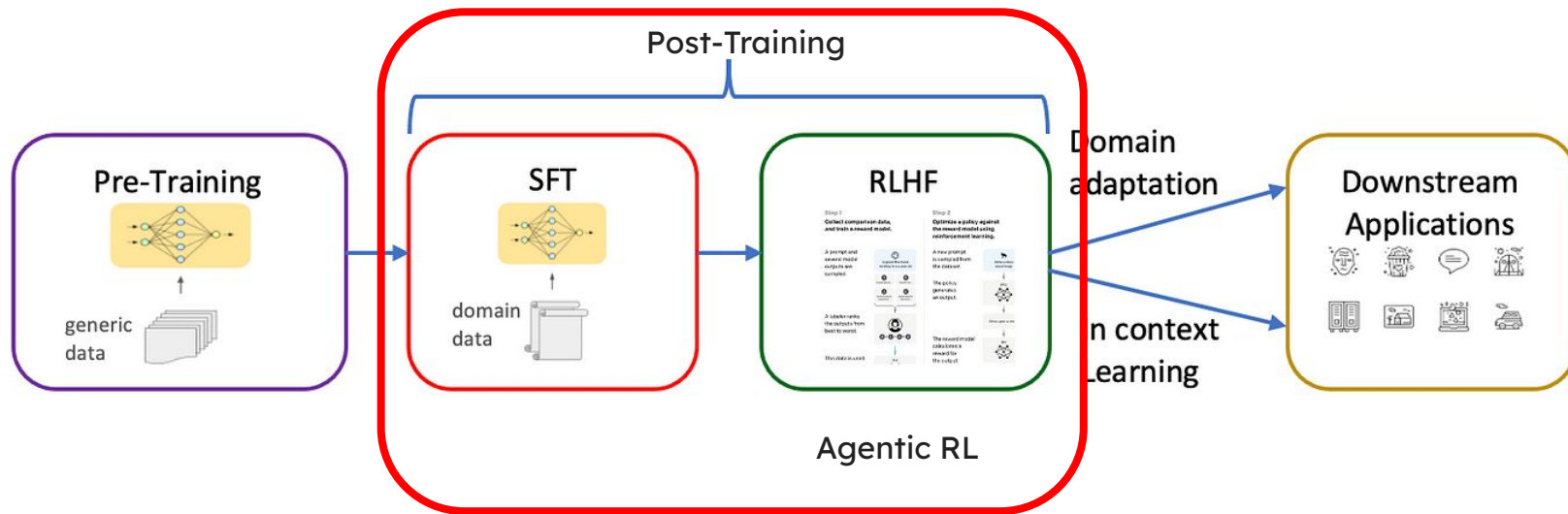
LLM Training



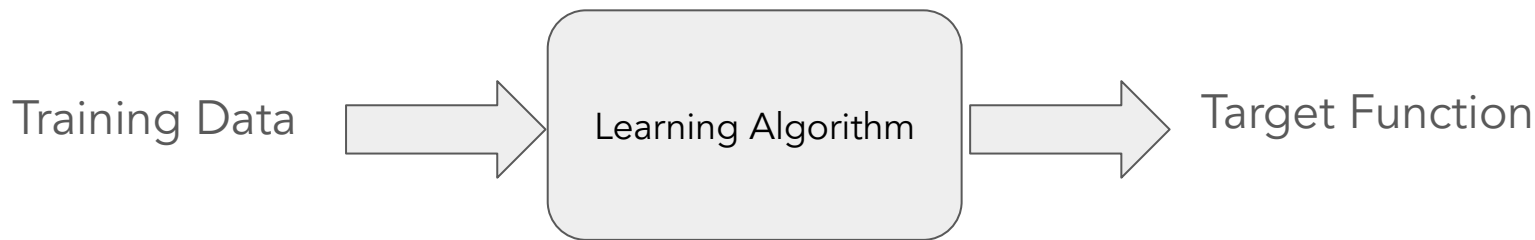
LLM Training



LLM Training



Pretraining of (Large) Language Models



$$D = \{X^i\}_{i=1}^n, \quad X^i = \{x_j^i\}_{j=1}^t$$

$$p(X)$$

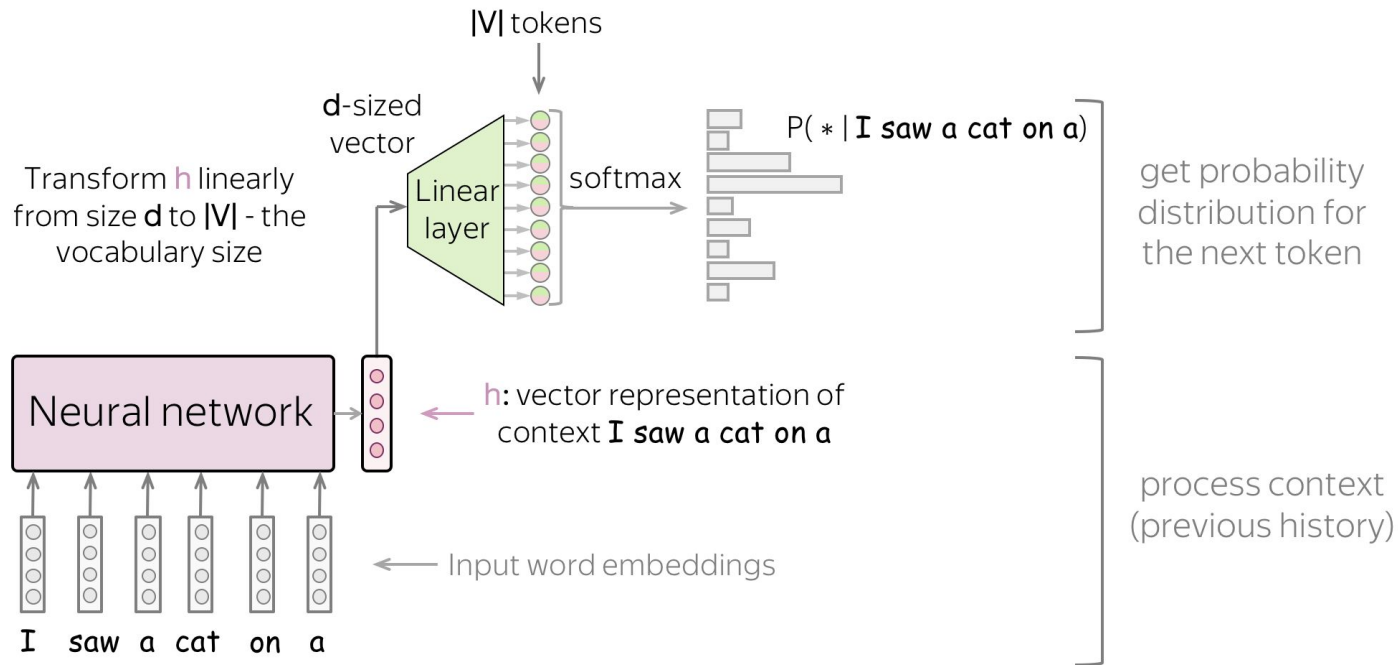
1. Build probabilistic models
Categorical Distribution + Autoregressive +
RNN->Transformer
2. Derive loss function (by MLE or MAP....)
MLE
3. Select optimizer
Stochastic Gradient Descent

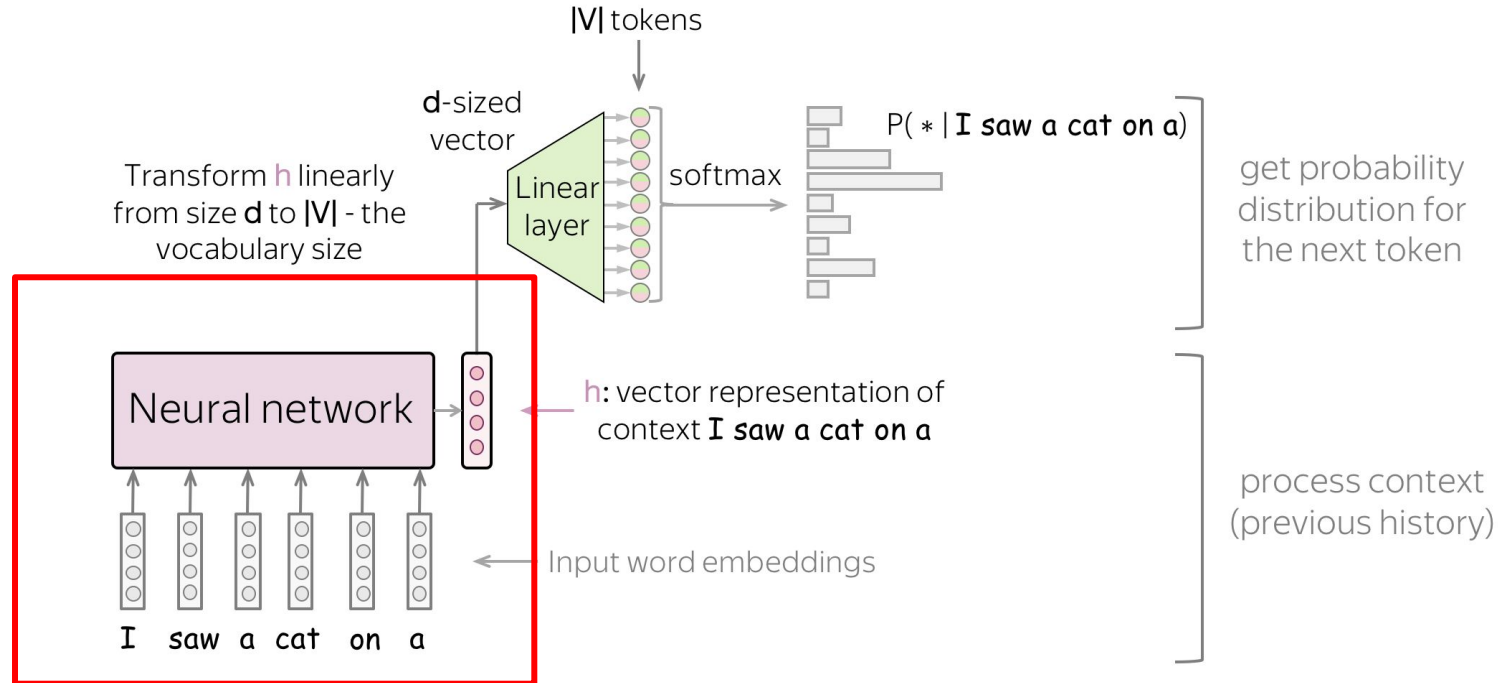
(Large) Language Models

$$\begin{aligned} p(X) &= p(\{x_j\}_{j=1}^t) \\ &= \prod_{j=1}^t p(x_j | x_{<j}) \\ &= \prod_{j=1}^t \frac{\exp(W_{x_j} \phi(x_{<j}))}{\sum_{l=1}^V \exp(W_l \phi(x_{<j}))} \end{aligned}$$

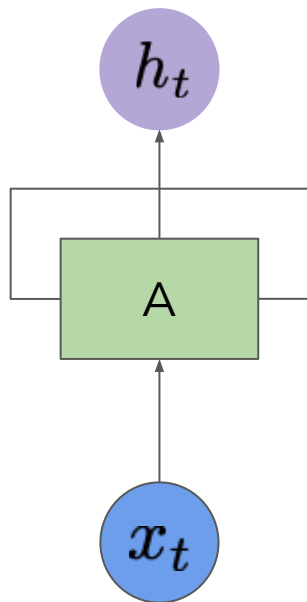
Recursive Neural Network in (Large) Language Models

$$\begin{aligned} p(X) &= p(\{x_j\}_{j=1}^t) && O(|V|^T) \\ &= \prod_{j=1}^t p(x_j | x_{<j}) \\ &= \prod_{j=1}^t \frac{\exp(W_{x_j} \phi(x_{<j}))}{\sum_{l=1}^V \exp(W_l \phi(x_{<j}))} \end{aligned} \quad \text{RNN}$$

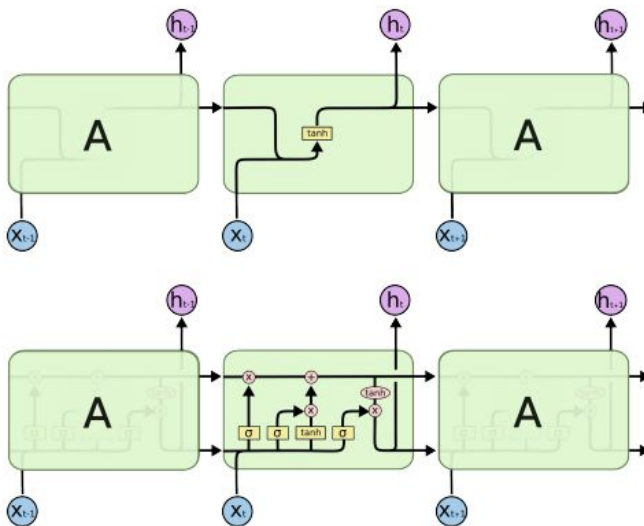




RNN Cell



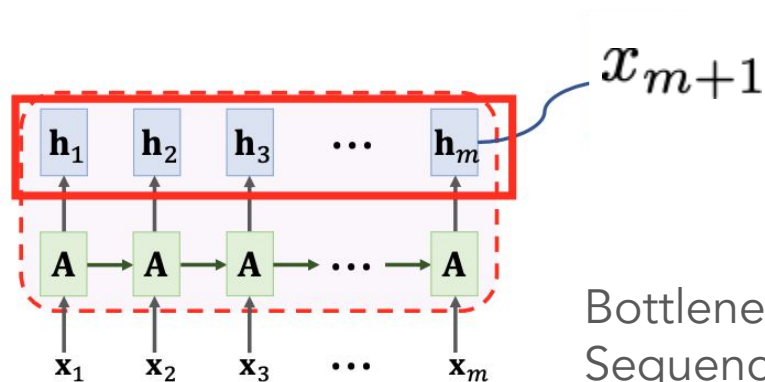
$$\frac{\exp(W_{x_j} \phi(x_{<j}))}{\sum_{l=1}^V \exp(W_l \phi(x_{<j}))}$$



Simple RNN

LSTM

Bottleneck in RNN



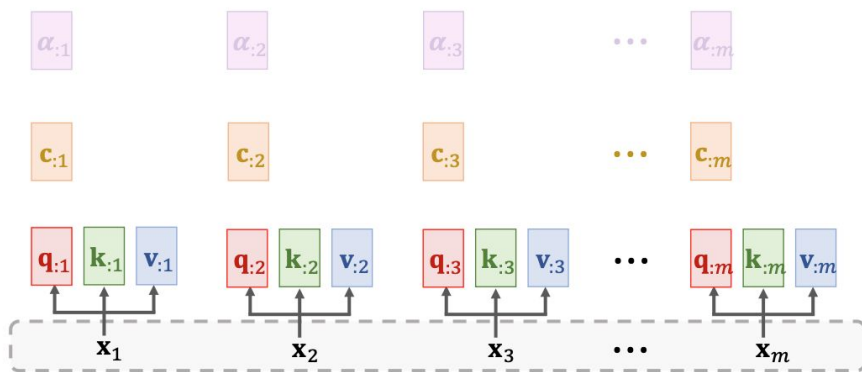
$$h_j = \phi(x_{<j})$$

$$\frac{\exp(W_{x_j} \phi(x_{<j}))}{\sum_{l=1}^V \exp(W_l \phi(x_{<j}))}$$

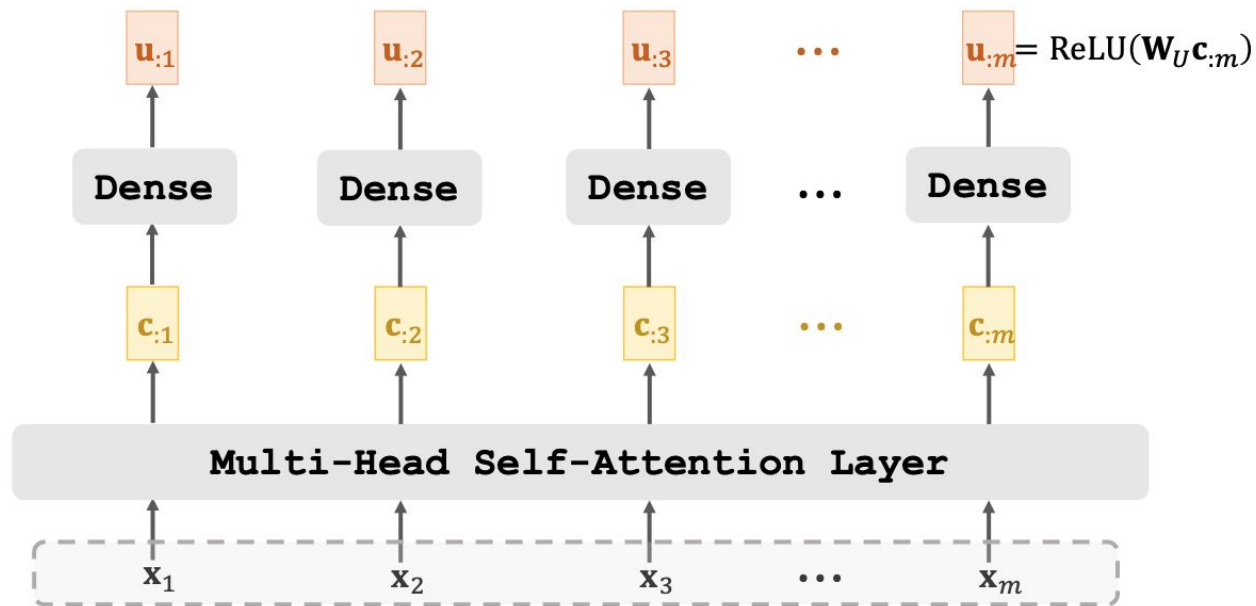
Bottleneck:
Sequences bottlenecked through a
fixed-sized vector.

Self-Attention

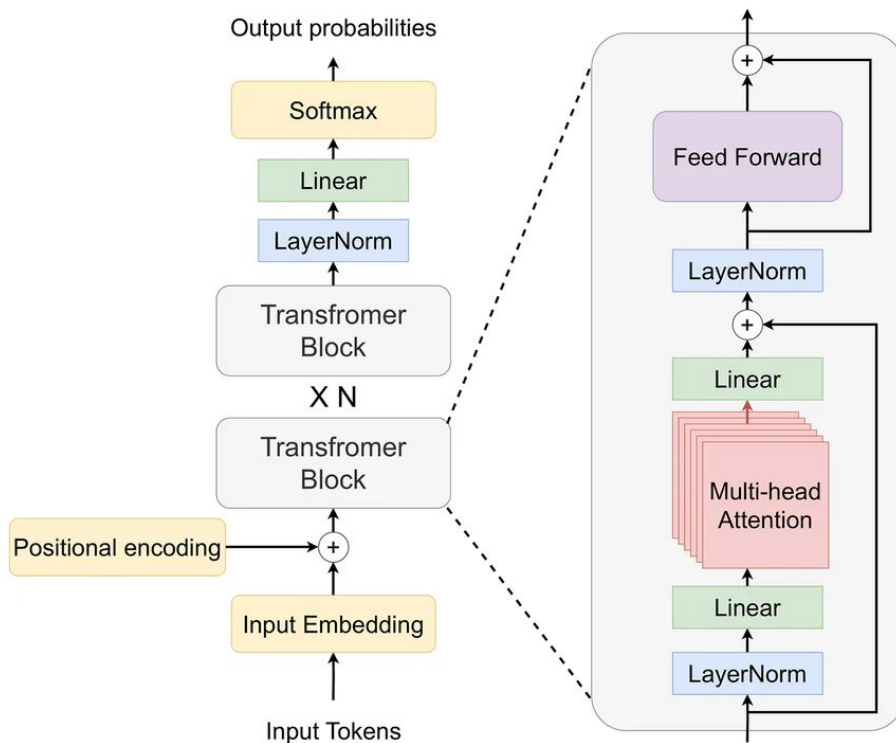
Context vector: $\mathbf{c}_{:1} = \alpha_{11}\mathbf{v}_{:1} + \dots + \alpha_{m1}\mathbf{v}_{:m} = \mathbf{V}\boldsymbol{\alpha}_{:1}.$



Multi-Headed Self-Attention



Transformer: Multi-Layer Multi-Headed Self-Attention!



Is Pretraining Enough for (L)LMs?

What we have:

$$p(X) = p(\{x_j\}_{j=1}^t) = \prod_{j=1}^t p(x_j | x_{<j})$$

Generating texts unconditionally

Is Pretraining Enough for (L)LMs?

What we have:

$$p(X) = p(\{x_j\}_{j=1}^t) = \prod_{j=1}^t p(x_j | x_{<j})$$

Generating texts unconditionally

Answer Question? Generating texts condition on answer

Is Pretraining Enough for (L)LMs?

What we have:

$$p(X) = p(\{x_j\}_{j=1}^t) = \prod_{j=1}^t p(x_j | x_{<j})$$

Generating texts unconditionally

Answer Question? Generating texts condition on answer

$$p(Y|X) = \prod_{l=1}^L p(y_l | y_{<l}, X)$$

Is Pretraining Enough for (L)LMs?

What we have:

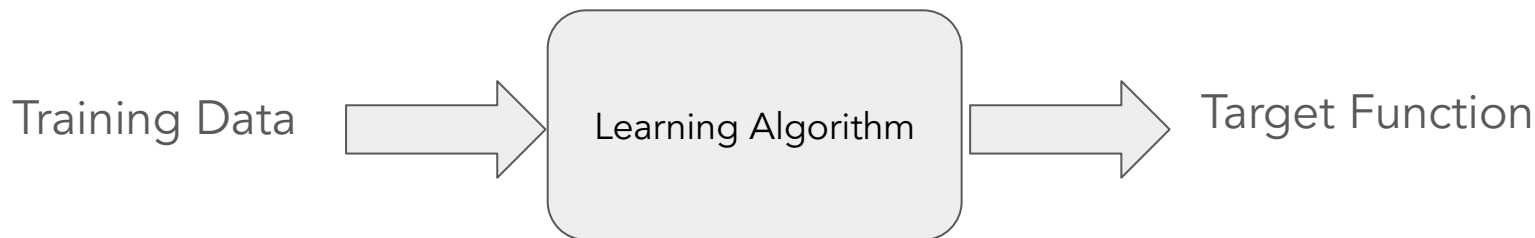
$$p(X) = p(\{x_j\}_{j=1}^t) = \prod_{j=1}^t p(x_j | x_{<j})$$

Generating texts unconditionally

Answer Question? Generating texts condition on answer

$$p(Y|X) = \prod_{l=1}^L p(y_l | y_{<l}, X) \quad \text{Never be trained}$$

Supervised Fine-tuning (Instruction Fine-tuning)



$$D = \{X^i, Y^i\}_{i=1}^n, \quad X^i = \{x_j^i\}_{j=1}^t, \quad Y^i = \{y_l^i\}_{l=1}^L$$

$$p(Y|X)$$

MATH Dataset (Ours)

Problem: Tom has a red marble, a green marble, a blue marble, and three identical yellow marbles. How many different groups of two marbles can Tom choose?

Solution: There are two cases here: either Tom chooses two yellow marbles (1 result), or he chooses two marbles of different colors ($\binom{4}{2} = 6$ results). The total number of distinct pairs of marbles Tom can choose is $1 + 6 = \boxed{7}$.

Problem: If $\sum_{n=0}^{\infty} \cos^{2n} \theta = 5$, what is $\cos 2\theta$?

Solution: This geometric series is $1 + \cos^2 \theta + \cos^4 \theta + \dots = \frac{1}{1 - \cos^2 \theta} = 5$. Hence,

$$\cos^2 \theta = \frac{4}{5}. \text{ Then } \cos 2\theta = 2 \cos^2 \theta - 1 = \boxed{\frac{3}{5}}.$$

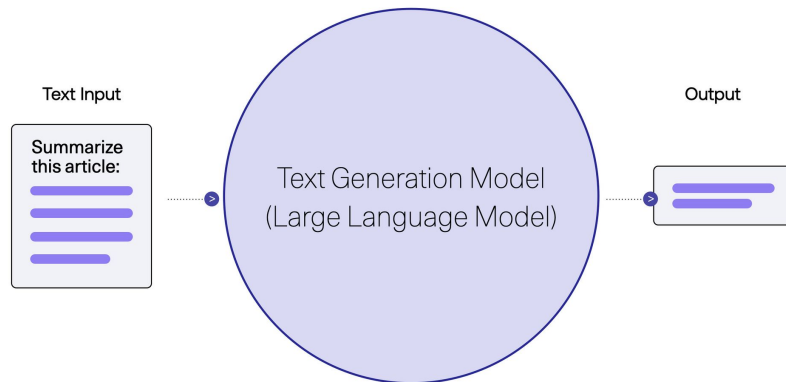
Problem: The equation $x^2 + 2x = i$ has two complex solutions. Determine the product of their real parts.

Solution: Complete the square by adding 1 to each side.

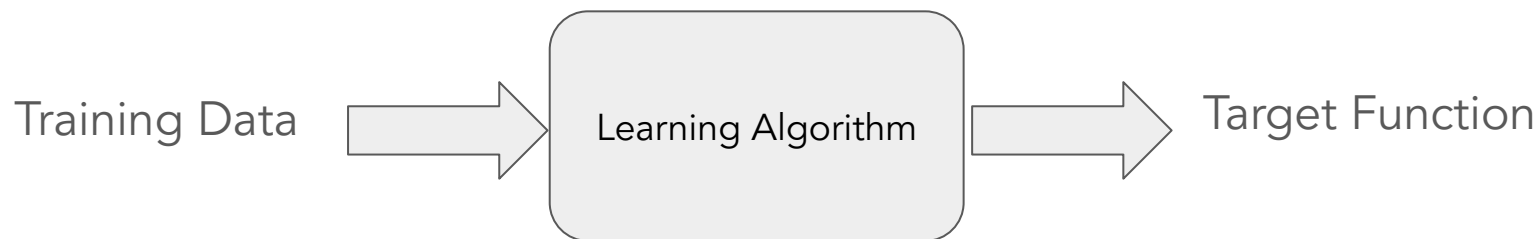
Then $(x+1)^2 = 1+i = e^{\frac{\pi}{4}} \sqrt{2}$, so $x+1 = \pm e^{\frac{\pi}{8}} \sqrt{2}$. The desired product is then

$$\left(-1 + \cos\left(\frac{\pi}{8}\right) \sqrt{2}\right) \left(-1 - \cos\left(\frac{\pi}{8}\right) \sqrt{2}\right) =$$

$$1 - \cos^2\left(\frac{\pi}{8}\right) \sqrt{2} = 1 - \frac{(1 + \cos(\frac{\pi}{4}))}{2} \sqrt{2} = \boxed{\frac{1 - \sqrt{2}}{2}}$$



Supervised Fine-tuning (Instruction Fine-tuning)



$$D = \{X^i, Y^i\}_{i=1}^n, \quad X^i = \{x_j^i\}_{j=1}^t, \quad Y^i = \{y_l^i\}_{l=1}^L$$

$$p(Y|X)$$

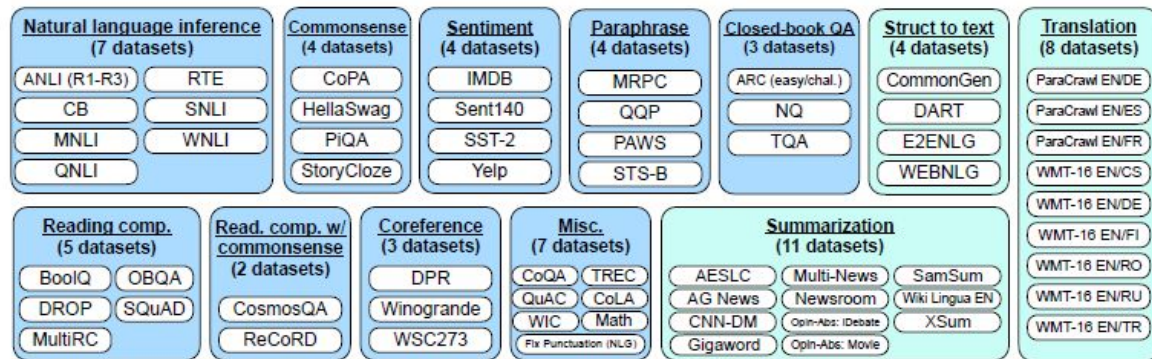
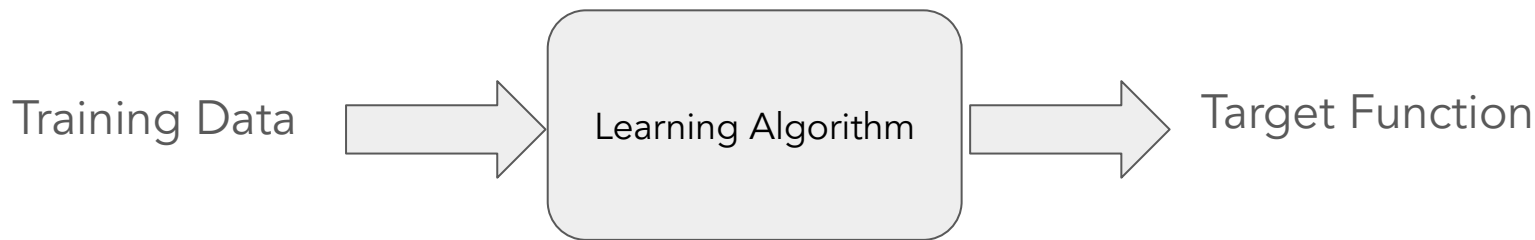


Figure 3: Datasets and task clusters used in this paper (NLU tasks in blue; NLG tasks in teal).

Supervised Fine-tuning (Instruction Fine-tuning)

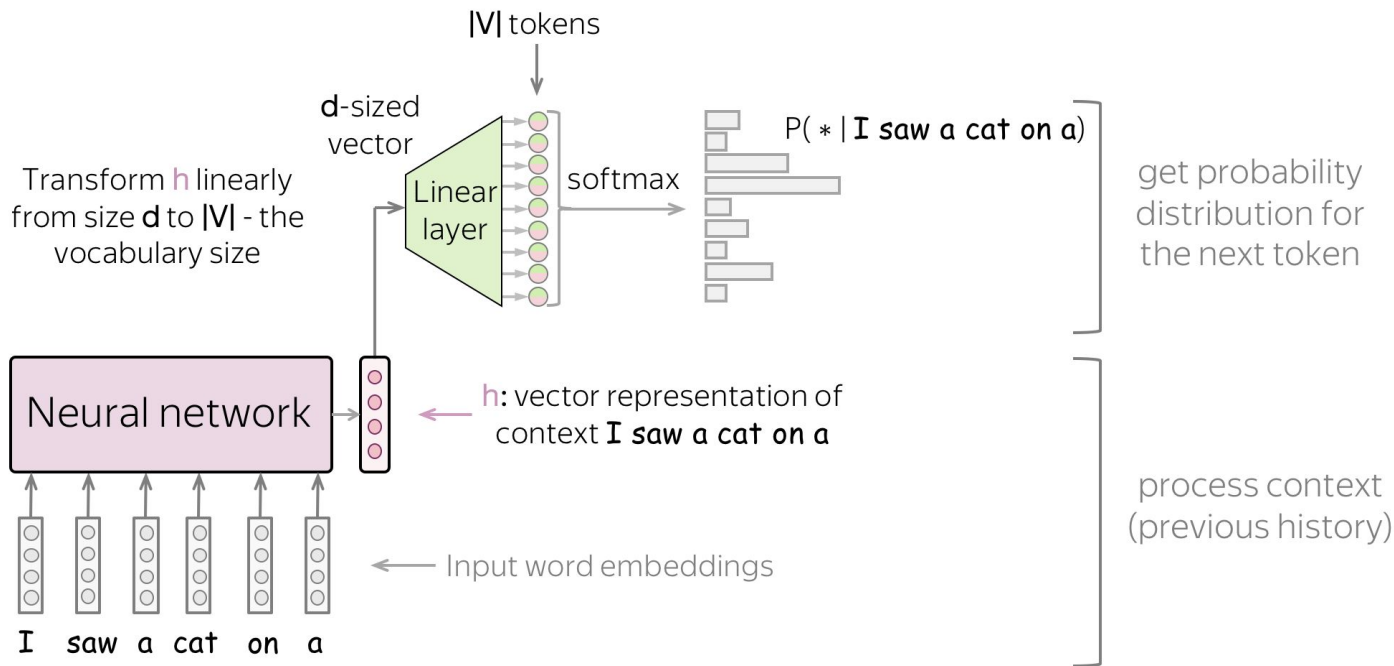


$$D = \{X^i, Y^i\}_{i=1}^n, \quad X^i = \{x_j^i\}_{j=1}^t, \quad Y^i = \{y_l^i\}_{l=1}^L$$

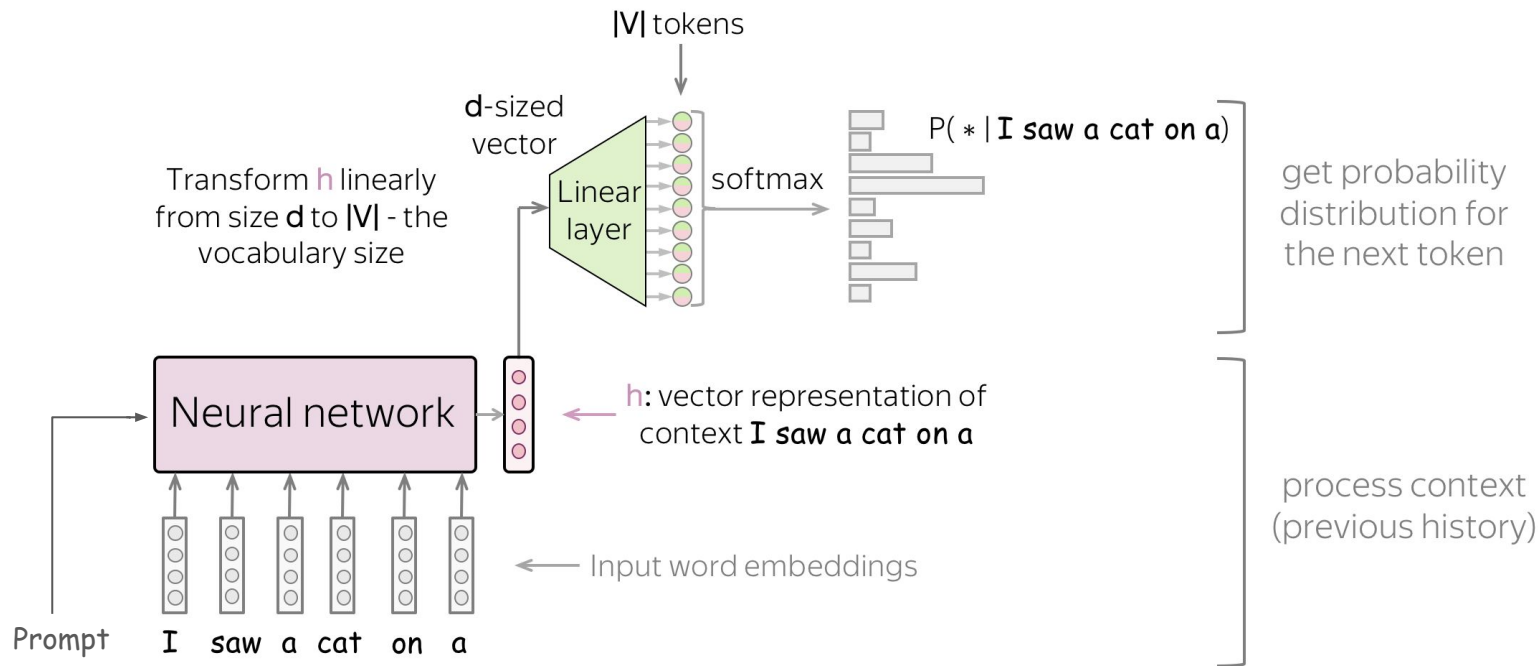
$$p(Y|X)$$

1. Build probabilistic models
Fixed the same as in pretraining:
Categorical Distribution + Autoregressive +
RNN/Transformer
2. Derive loss function (by MLE or MAP...)
3. Select optimizer

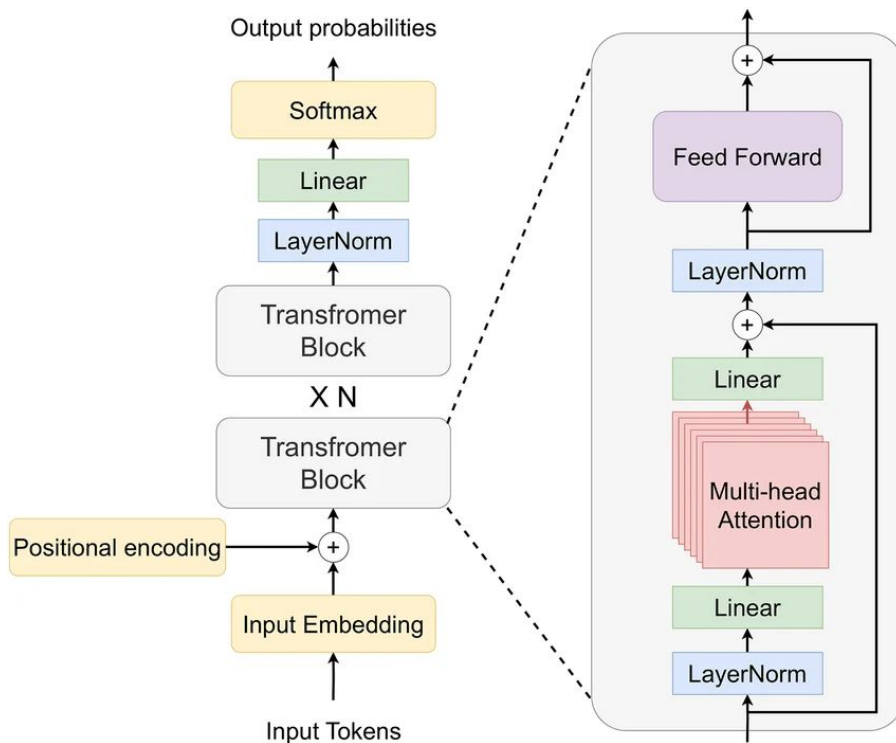
LLM Architecture Naturally Fits



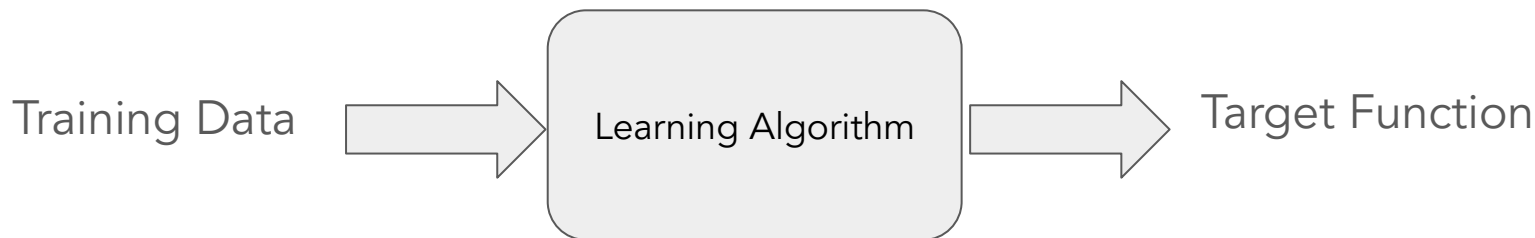
LLM Architecture Naturally Fits



Transformer: Multi-Layer Multi-Headed Self-Attention!



Supervised Fine-tuning (Instruction Fine-tuning)



$$D = \{X^i, Y^i\}_{i=1}^n, \quad X^i = \{x_j^i\}_{j=1}^t, \quad Y^i = \{y_l^i\}_{l=1}^L$$

$$p(X|Y)$$

1. Build probabilistic models
2. Derive loss function (by MLE or MAP....)
MLE
3. Select optimizer

MLE for Supervised Fine-tuning (SFT)

- Given all input data $D = \{X^i, Y^i\}_{i=1}^n$, $X^i = \{x_j^i\}_{j=1}^t$, $Y^i = \{y_l^i\}_{l=1}^L$

$$p(Y | X) = \prod_{l=1}^L p(y_l | y_{<l}, X)$$

- Log-likelihood

$$\begin{aligned}\ell(\phi) &= \sum_{i=1}^n p(Y^i | X^i; \phi) = \sum_{i=1}^n \sum_{l=1}^L \log(y_l^i | y_{<l}^i, X^i; \phi) \\ &= \sum_{i=1}^n \sum_{l=1}^L \log \frac{\exp(W_{y_l^i} \phi(y_{<l}^i | X^i))}{\sum_{v=1}^V \exp(W_v \phi(y_{<l}^i | X^i))} \\ &= \sum_{i=1}^n \sum_{l=1}^L W_{y_l^i} \phi(y_{<l}^i | X^i) - \sum_{i=1}^n \sum_{j=1}^t \log \sum_{v=1}^V \exp(W_v \phi(y_{<l}^i | X^i))\end{aligned}$$

MLE for Supervised Fine-tuning (SFT)

- Given all input data $D = \{X^i, Y^i\}_{i=1}^n$, $X^i = \{x_j^i\}_{j=1}^t$, $Y^i = \{y_l^i\}_{l=1}^L$

$$p(Y | X) = \prod_{l=1}^L p(y_l | y_{<l}, X)$$

- Log-likelihood

$$\begin{aligned}\ell(\phi) &= \sum_{i=1}^n p(Y^i | X^i; \phi) = \sum_{i=1}^n \sum_{l=1}^L \log(y_l^i | y_{<l}^i, X^i; \phi) \\ &= \sum_{i=1}^n \sum_{l=1}^L \log \frac{\exp(W_{y_l^i} \phi(y_{<l}^i | X^i))}{\sum_{v=1}^V \exp(W_v \phi(y_{<l}^i | X^i))} \\ &= \sum_{i=1}^n \sum_{l=1}^L W_{y_l^i} \phi(y_{<l}^i | X^i) - \sum_{i=1}^n \sum_{j=1}^t \log \sum_{v=1}^V \exp(W_v \phi(y_{<l}^i | X^i))\end{aligned}$$

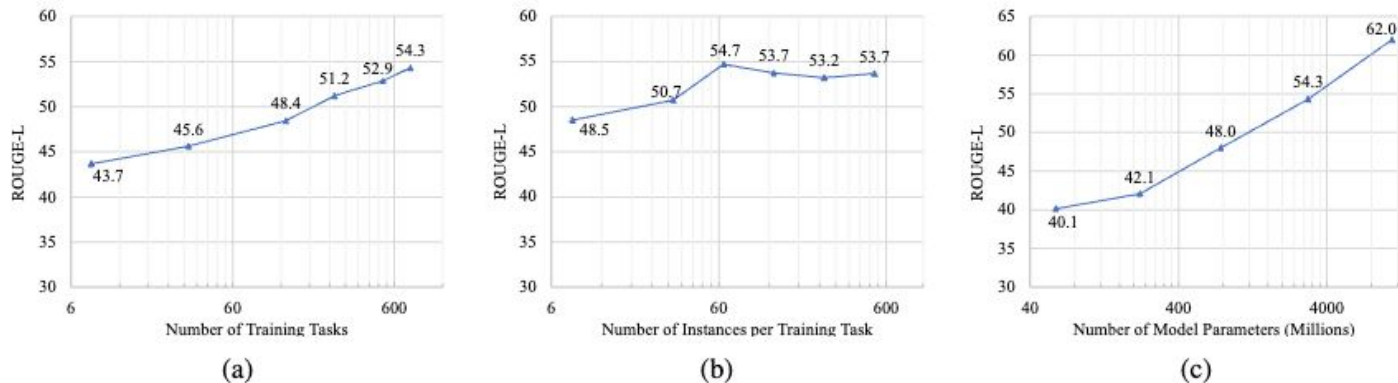
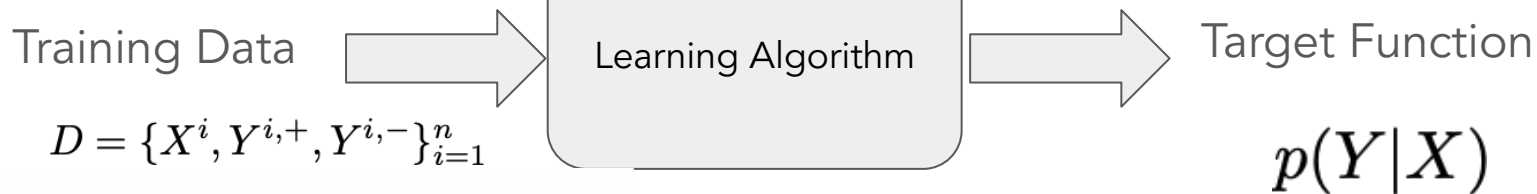


Figure 5: Scaling trends of models performance (§7.1) as a function of (a) the number of training tasks; (b) the number of instances per training task; (c) model sizes. x -axes are in log scale. The **linear growth of model performance with exponential increase in observed tasks and model size** is a promising trend. Evidently, the performance gain from more instances is limited.

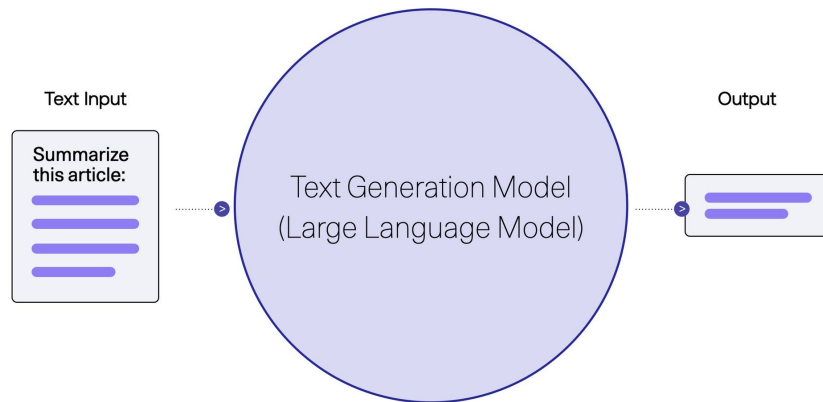
RLHF for (Large) Language Models



What are the key benefits of using Reinforcement Learning from Human Feedback (RLHF) for dataset collection in the context of Large Language Model (LLM) generation?

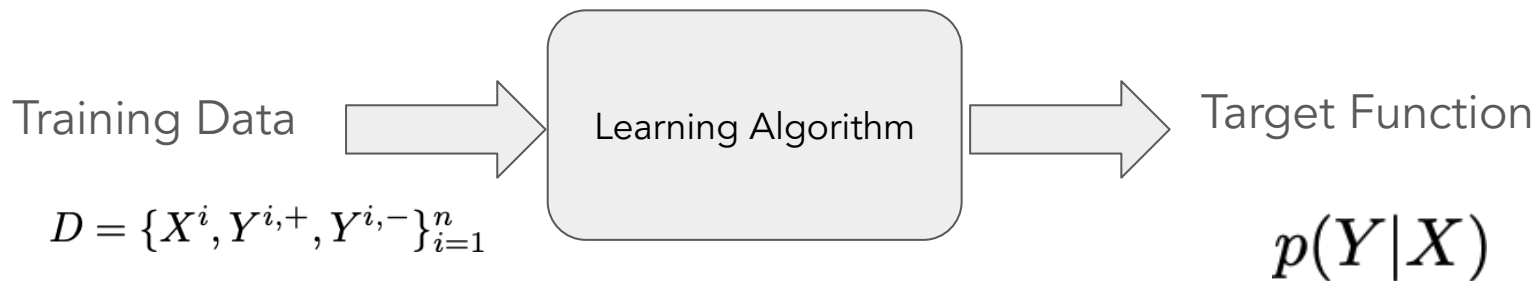
Reinforcement Learning from Human Feedback (RLHF) for dataset collection in Large Language Model (LLM) generation provides key benefits such as improved model performance through direct optimization, better alignment with human values by incorporating human feedback, and the ability to iteratively refine the model based on user interactions, resulting in a more user-friendly and efficient language model.

Using reinforcement Learning from Human Feedback (RLHF) for dataset collection in Large Language Model (LLM) generation offers advantages such as enhanced model capabilities by optimizing for desired outcomes, greater adaptability to human preferences through the inclusion of human feedback, and the opportunity to continuously improve the model based on user experiences, ultimately leading to a more effective and responsive language model.



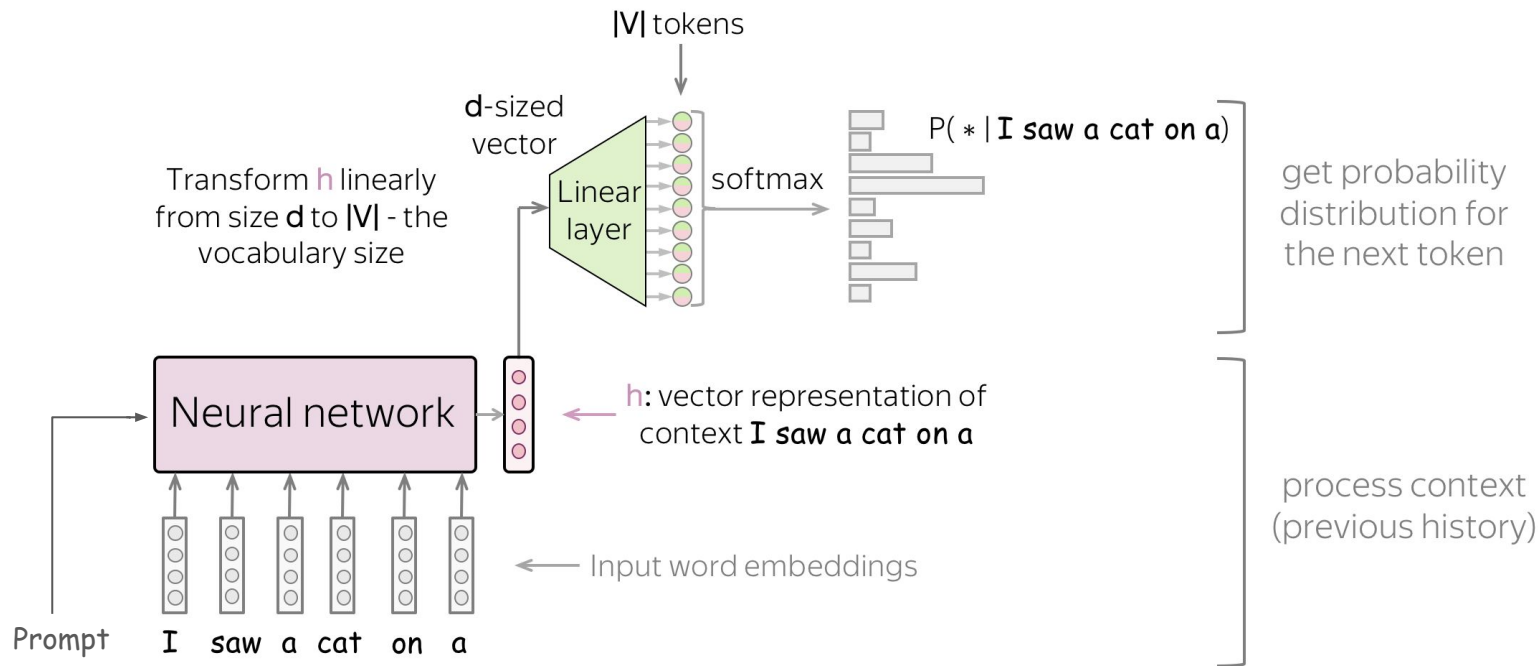
Preference Data: Prompt & Positive/ Negative Answers

RLHF for (Large) Language Models

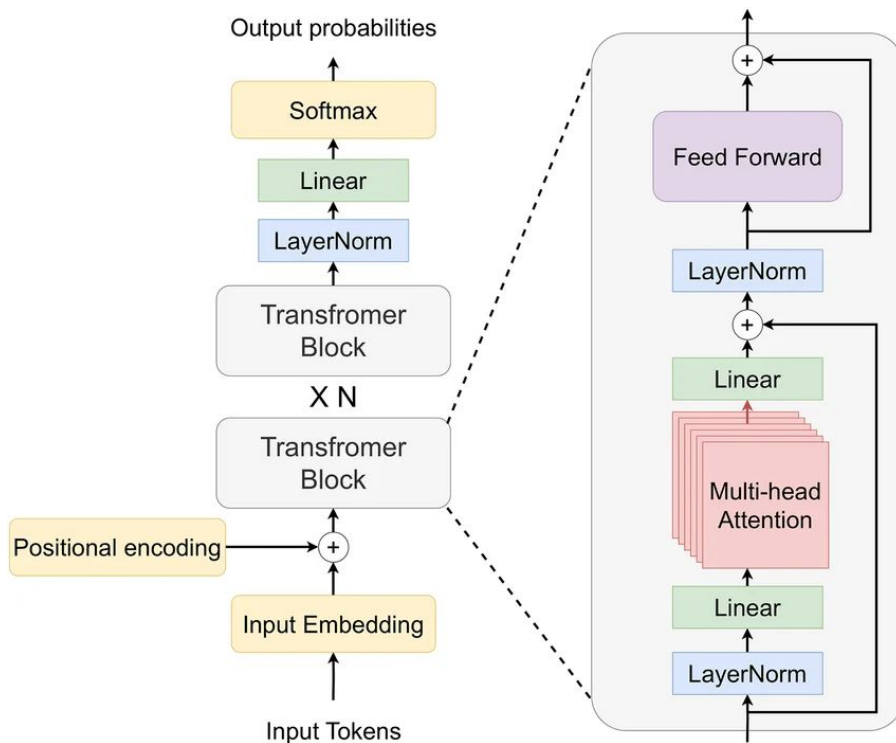


1. Build probabilistic models
Fixed the same as in pretraining:
Categorical Distribution + Autoregressive +
RNN/Transformer
2. Derive loss function (by MLE or MAP....)
3. Select optimizer

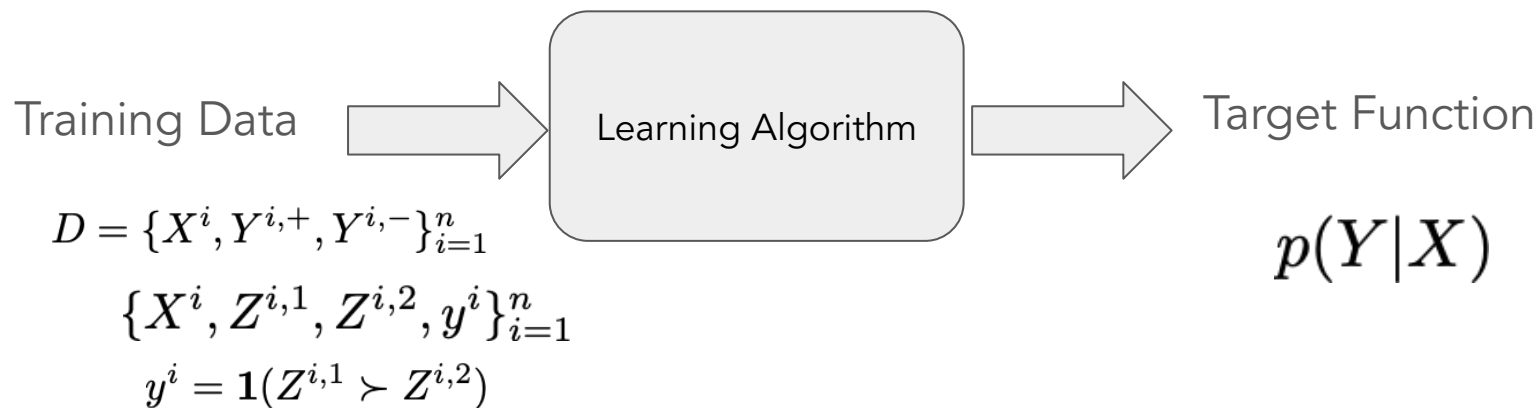
LLM Architecture Naturally Fits



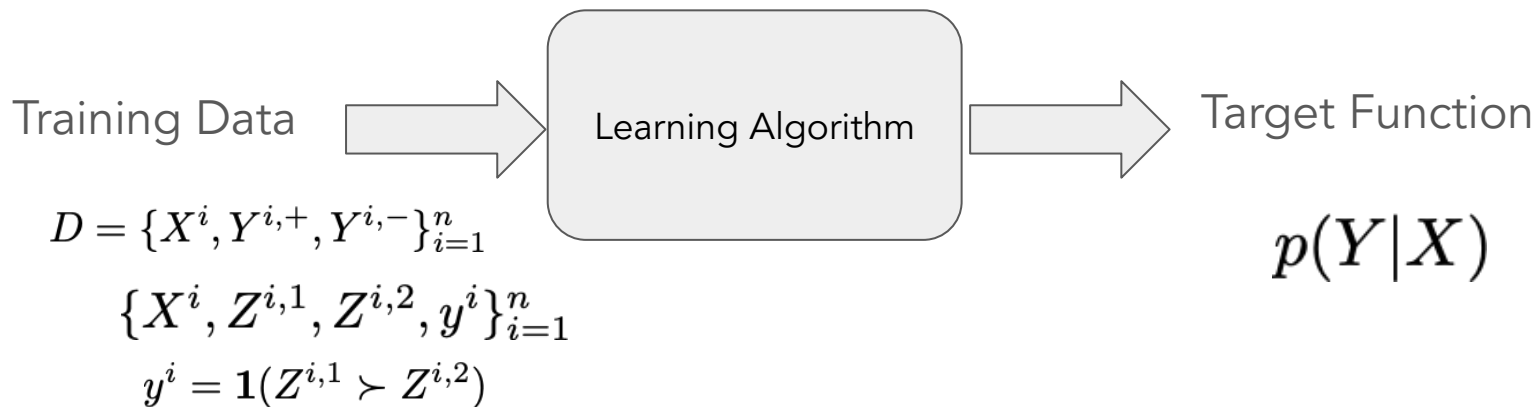
Transformer: Multi-Layer Multi-Headed Self-Attention!



RLHF for (Large) Language Models



RLHF for (Large) Language Models

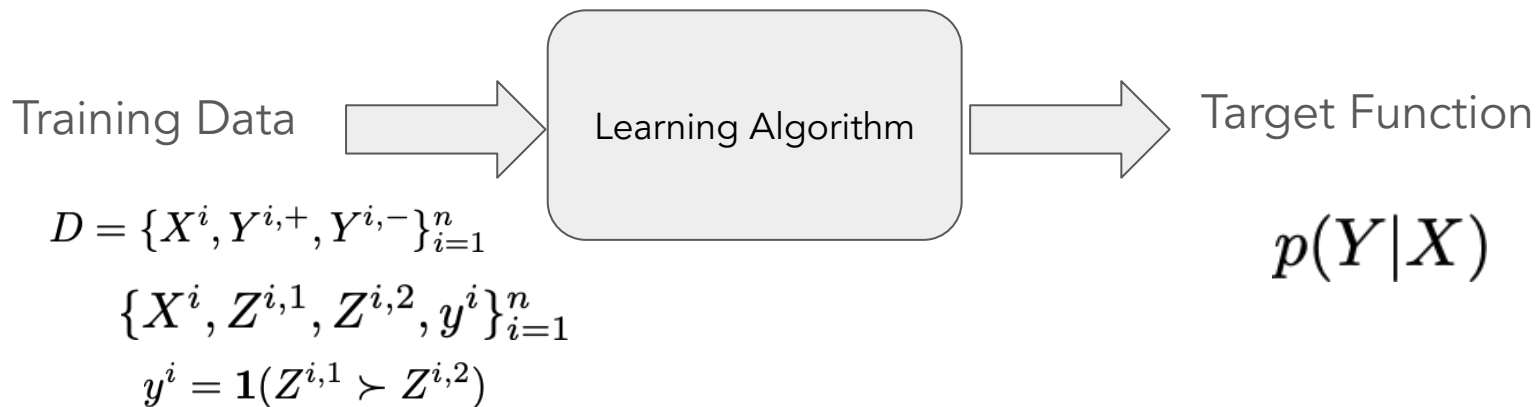


Proximal Policy Optimization (PPO)

Direct Preference Optimization (DPO)

Group Relative Policy Optimization (GRPO),

RLHF for (Large) Language Models

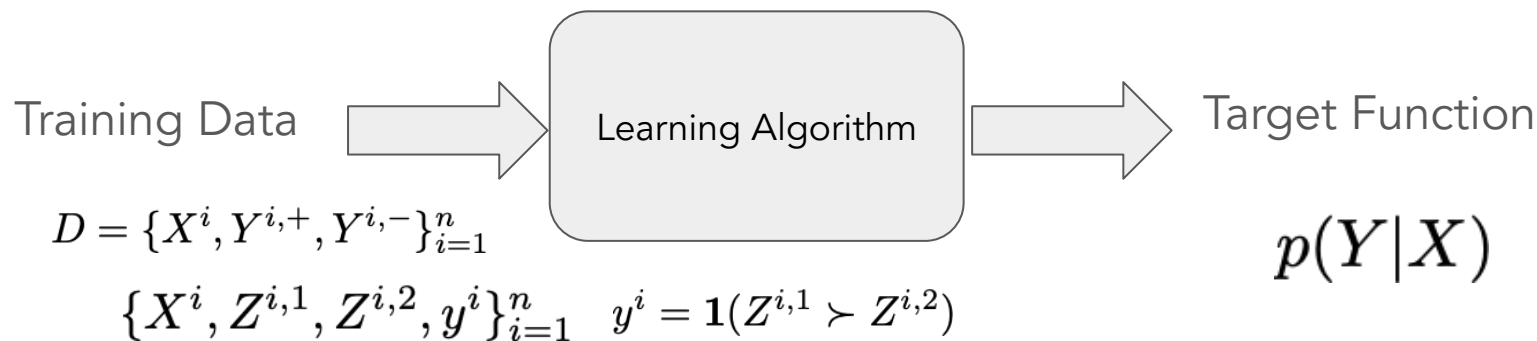


Proximal Policy Optimization (PPO)

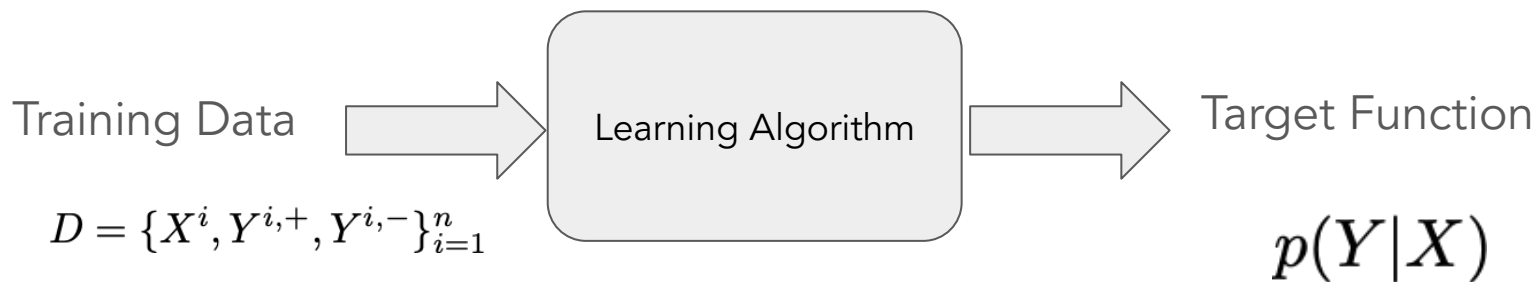
Direct Preference Optimization (DPO) -> reduce to classification

Group Relative Policy Optimization (GRPO),

RLHF for (Large) Language Models



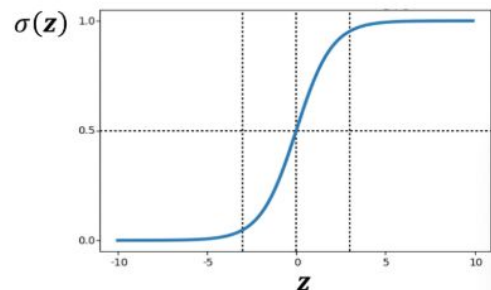
RLHF for (Large) Language Models



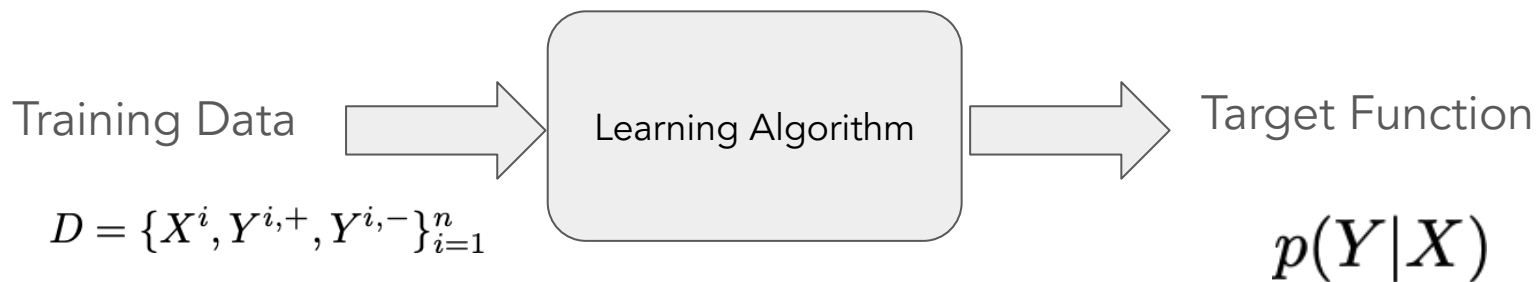
$$\{X^i, Z^{i,1}, Z^{i,2}, y^i\}_{i=1}^n \quad y^i = \mathbf{1}(Z^{i,1} \succ Z^{i,2})$$

$$\sigma(\mathbf{z}) = \frac{1}{1 + e^{-z}} = \frac{e^z}{1 + e^z}$$

$$p(Z^1 \succ Z^2 | X) = \sigma\left(\log \frac{p(Z^1|x)}{p_{\text{ref}}(Z^1|x)} - \log \frac{p(Z^2|x)}{p_{\text{ref}}(Z^2|x)}\right)$$



RLHF for (Large) Language Models



1. Build probabilistic models
2. Derive loss function (by MLE or MAP....)
MLE
3. Select optimizer

MLE for Preference Learning - Direct Preference Optimization (DPO)

- Given all input data $D = \{X^i, Y^{i,+}, Y^{i,-}\}_{i=1}^n$

$$\begin{aligned} \{X^i, Z^{i,1}, Z^{i,2}, y^i\}_{i=1}^n \\ y^i = \mathbf{1}(Z^{i,1} \succ Z^{i,2}) \end{aligned} \quad \Rightarrow \quad p(Z^1 \succ Z^2 | X) = \sigma \left(\log \frac{p(Z^1 | x)}{p_{\text{ref}}(Z^1 | x)} - \log \frac{p(Z^2 | x)}{p_{\text{ref}}(Z^2 | x)} \right)$$

- Log-likelihood

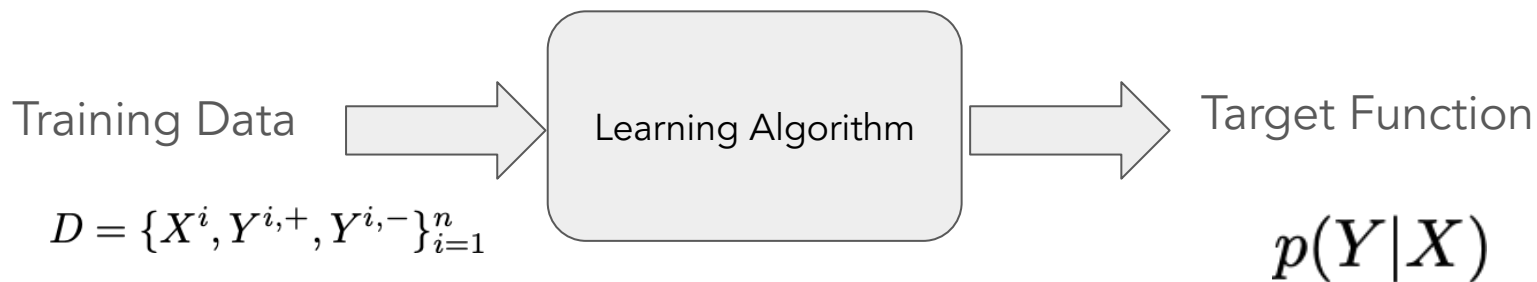
$$\begin{aligned} \ell(\phi) &= \sum_{i=1}^n y^i \log p(Z^{i,1} \succ Z^{i,2} | X^i; \phi) \\ &= \sum_{i=1}^n \log \sigma \left(\log \frac{p_{\phi}(Y^{i,+} | X^i)}{p_{\text{ref}}(Y^{i,+} | X^i)} - \log \frac{p_{\phi}(Y^{i,-} | X^i)}{p_{\text{ref}}(Y^{i,-} | X^i)} \right) \end{aligned}$$

Online vs. Offline Samples

$$\begin{aligned}\ell(\phi) &= \sum_{i=1}^n y^i \log p(Z^{i,1} \succ Z^{i,2} \mid X^i; \phi) \\ &= \sum_{i=1}^n \log \sigma \left(\log \frac{p_{\phi}(Y^{i,+} \mid X^i)}{p_{\text{ref}}(Y^{i,+} \mid X^i)} - \log \frac{p_{\phi}(Y^{i,-} \mid X^i)}{p_{\text{ref}}(Y^{i,-} \mid X^i)} \right)\end{aligned}$$

Where the samples Y comes from?

RLHF for (Large) Language Models



1. Build probabilistic models
2. Derive loss function (by MLE or MAP....)
3. Select optimizer

Stochastic Gradient Descent

Proximal Policy Optimization (PPO) vs. DPO

1, Learn a reward model

$$\text{loss}(\theta) = -\frac{1}{\binom{K}{2}} E_{(x, y_w, y_l) \sim D} [\log(\sigma(r_\theta(x, y_w) - r_\theta(x, y_l)))]$$

2, Policy Gradient

$$\max_{p(Y|X)} \sum_{X \sim D} \left(\mathbb{E}_{p(Y|X)}[r(X, Y)] - \lambda KL(p(Y|X) || p_{\text{ref}}(Y|X)) \right)$$

Proximal Policy Optimization (PPO) vs. DPO

1, Learn a reward model

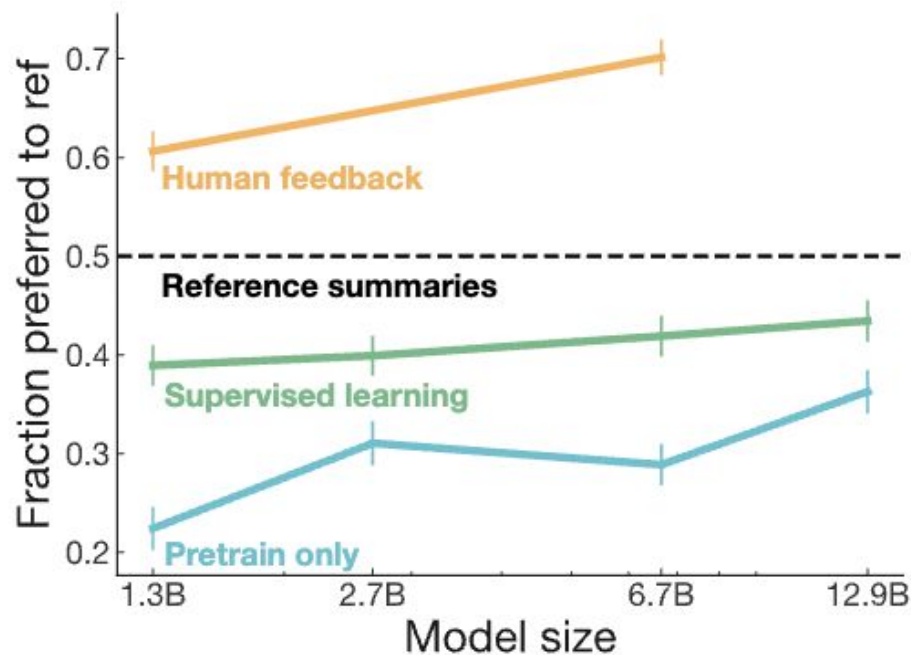
$$E_{(x, y_w, y_l) \sim D} [\log (\sigma (r_{\theta} (x, y_w) - r_{\theta} (x, y_l)))]$$

2, Policy Gradient

$$\max_{p(Y|X)} \sum_{X \sim D} \left(\mathbb{E}_{p(Y|X)} [r(X, Y)] - \lambda KL(p(Y|X) || p_{\text{ref}}(Y|X)) \right)$$

Naturally Online!

RLHF Performance

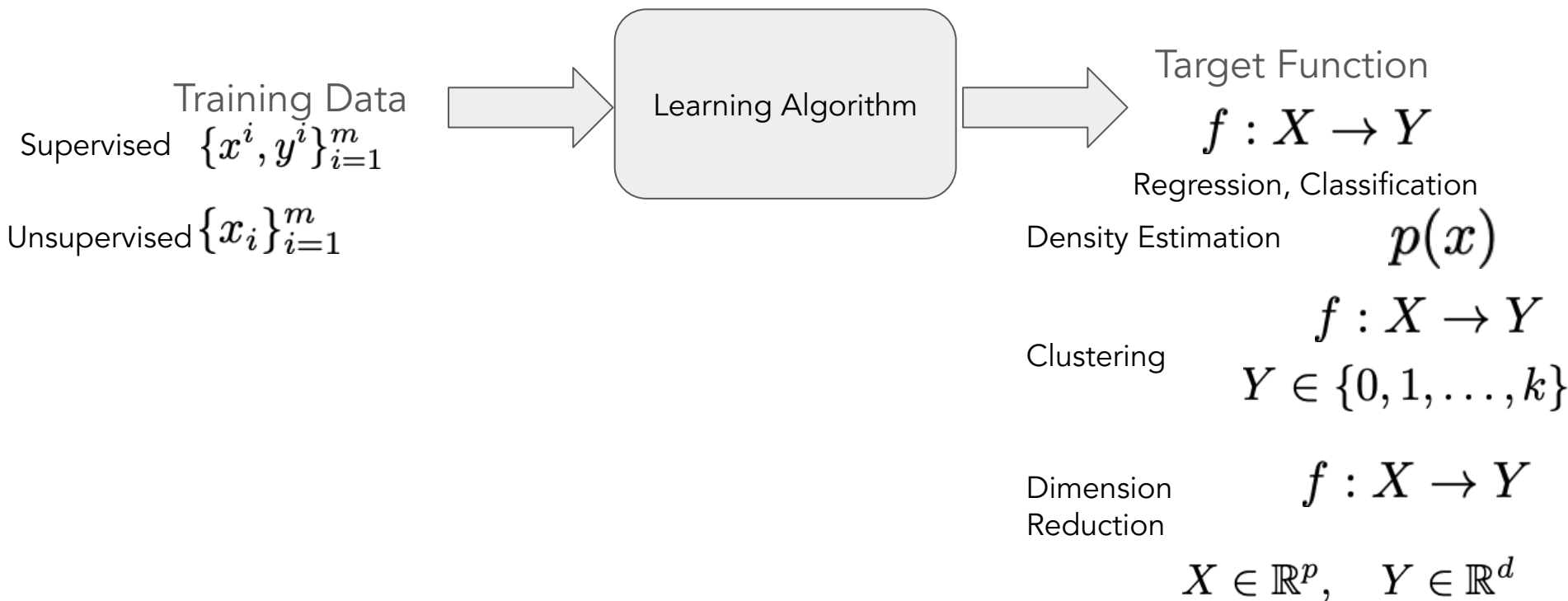


Stiennon, Nisan, et al. "Learning to summarize with human feedback." *Advances in neural information processing systems* 33 (2020): 3008-3021.

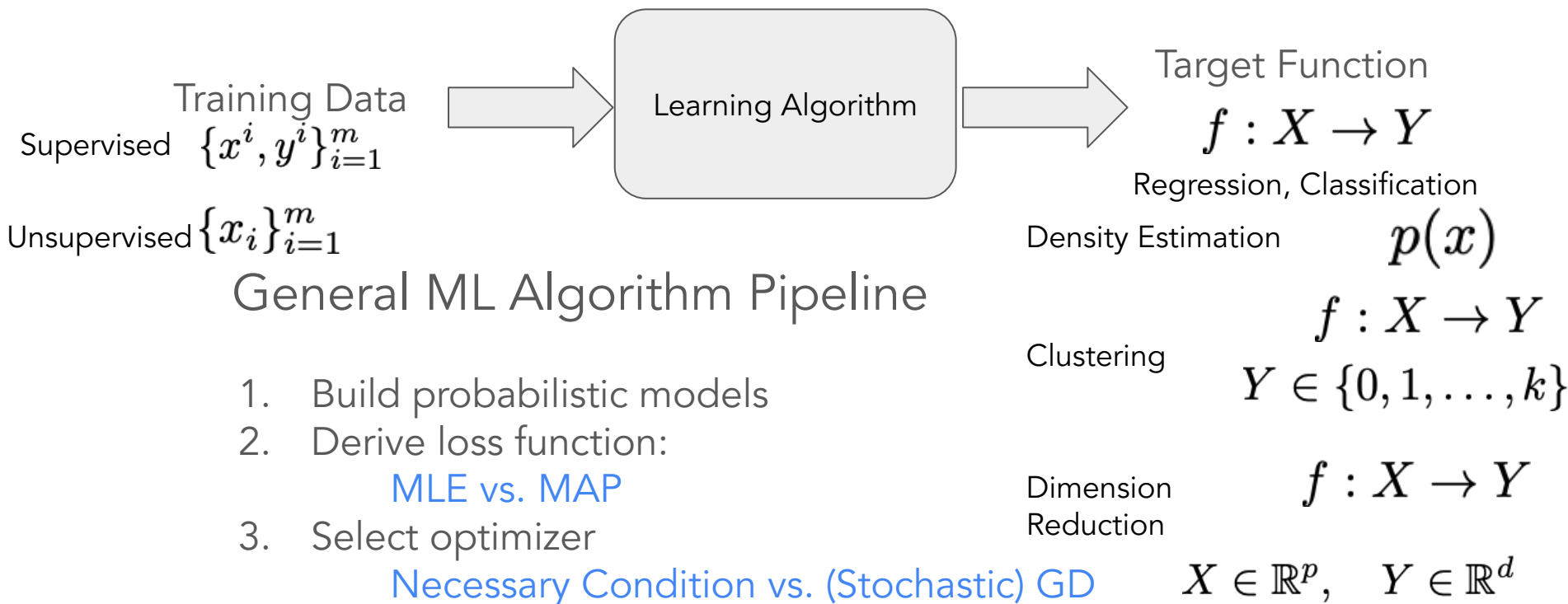
Summary

- Pretraining of LLM is not enough
- Post-training of LLM is necessary:
 - Supervised Fine-tuning
 - Reinforcement Learning from Human Feedback
 - Online vs Offline data

Supervised Learning vs. Unsupervised Learning



Supervised Learning vs. Unsupervised Learning



Q&A

Thanks for
Attending in the
whole semester